

Your Research Paper Title

1st Your Name *Department*
University/Institution
City, Country
email@example.com

2nd Co-Author Name *Department*
University/Institution
City, Country
coauthor@example.com

Abstract

Keywords—keyword1, keyword2, keyword3, keyword4, keyword5

1 Introduction

This section introduces the background, motivation, and contributions of your research.

1.1 Background

Logic synthesis is a fundamental step in digital design automation, significantly impacting the Quality of Results (QoR) such as area, timing, and power consumption. Traditional logic synthesis frameworks, such as ABC [3], employ greedy local optimization approaches that apply incremental changes to small portions of the circuit. These methods, while effective, often limit the exploration space and may miss opportunities for significant improvements.

Recently, E-Syn [1] has emerged as a novel logic optimization framework that leverages equivalence graphs (e-graphs) for Boolean logic rewriting. E-Syn addresses the limitations of traditional AIG-based synthesis by maintaining equivalent classes of logic specifications, allowing exploration of a wider range of equivalent logic forms. A key innovation of E-Syn is its use of technology-aware cost functions obtained from machine learning models, specifically XGBoost regression, to bridge the gap between technology-independent logic cost and post-mapping QoR.

1.2 Motivation

In E-Syn, the extraction phase plays a critical role in selecting the optimal logic form from the e-graph based on cost estimates. The quality of these cost estimates directly impacts the final QoR of the synthesized circuit. The current E-Syn framework uses XGBoost regression models to predict post-mapping area and delay from technology-independent features. However, the accuracy of these regression models has a direct influence on the extraction quality. Estimation errors in the cost function can lead to suboptimal candidate selection during extraction, potentially missing better logic forms that would result in superior QoR.

To address this limitation, we propose to optimize the regression training process in E-Syn to reduce estimation errors. By improving the accuracy of the cost prediction models, we can enhance the quality of extraction decisions, leading to better final synthesis results. Specifically, we focus on:

- Improving the training data quality and feature engineering for the regression models
- Optimizing hyperparameters and model architectures to reduce prediction errors
- Enhancing the cost function accuracy to better reflect actual post-mapping QoR
- Validating that improved regression accuracy translates to better extraction quality and final synthesis results

1.3 Contributions

The main contributions of this work include:

- An analysis of the impact of regression model accuracy on E-Syn’s extraction quality and final synthesis results
- Investigation of how different features affect prediction errors in cost estimation models, identifying critical features that significantly influence estimation accuracy
- Comprehensive evaluation demonstrating that reduced estimation errors lead to better extraction decisions and improved QoR
- Comparison with multiple regression models (XGBoost, Random Forest, LightGBM, MLP, CatBoost) to identify the most suitable approach for cost estimation

1.4 Paper Organization

The rest of this paper is organized as follows: Section 2 formulates the problem of regression model optimization for E-Syn’s cost estimation. Section 3 presents our proposed methodology for improving regression training. Section 4 describes the experimental setup, including datasets, evaluation metrics, and baseline methods. Section 5 presents the experimental results and analysis. Section 6 discusses the findings and implications. Finally, Section 7 concludes the paper.

2 Problem Formulation

This section provides a detailed formulation of the problem being addressed.

As discussed in the introduction, the accuracy of regression models used for cost estimation in E-Syn directly impacts the quality of extraction decisions and the final synthesis results. To improve the overall performance of E-Syn, we focus on optimizing the regression training process from two perspectives:

1. **Modifying training models and methodologies:** We investigate different regression models and training approaches to reduce prediction errors. This includes exploring various machine learning algorithms (XGBoost, Random Forest, LightGBM, MLP, CatBoost) and optimizing their hyperparameters, training procedures, and model architectures to achieve better prediction accuracy for post-mapping area and delay estimation.
2. **Identifying useful features for prediction:** We analyze the impact of different features on prediction errors and identify which features are most critical for accurate cost estimation. This involves feature importance analysis, feature selection techniques, and understanding how different circuit characteristics contribute to the prediction accuracy of area and delay.

By addressing these two aspects, we aim to develop more accurate cost estimation models that can better guide the extraction phase in E-Syn, ultimately leading to improved Quality of Results (QoR) in logic synthesis.

3 Proposed Method

This section describes the proposed methodology in detail. Our approach focuses on two main aspects: exploring different regression models and enhancing feature engineering for cost estimation.

3.1 Regression Models

The original E-Syn framework primarily uses XGBoost for cost estimation. To improve prediction accuracy, we investigate multiple regression models and evaluate their performance. We select the following models based on their distinct characteristics and potential advantages:

- **XGBoost:** As the baseline model used in E-Syn, XGBoost is a gradient boosting framework that excels at handling non-linear relationships and feature interactions. It provides excellent performance for structured data and offers built-in regularization to prevent overfitting.
- **Random Forest:** We include Random Forest as an ensemble learning method that combines multiple decision trees. It is robust to overfitting and

can capture complex interactions between features through its bagging approach, providing a different perspective compared to gradient boosting methods.

- **LightGBM:** LightGBM is another gradient boosting framework that uses a leaf-wise tree growth strategy, making it faster and more memory-efficient than traditional gradient boosting methods. It is particularly suitable for large-scale datasets and can handle high-dimensional feature spaces effectively.
- **Multi-Layer Perceptron (MLP):** We incorporate MLP, a deep neural network model, to capture complex non-linear relationships that may not be easily modeled by tree-based methods. MLP can learn hierarchical feature representations and is capable of modeling intricate patterns in the data through its multiple hidden layers and non-linear activation functions. This makes it particularly valuable for discovering non-obvious relationships between circuit characteristics and post-mapping QoR.
- **CatBoost:** CatBoost is a gradient boosting algorithm designed to handle categorical features effectively and reduce overfitting through its ordered boosting technique. It provides robust performance with minimal hyperparameter tuning, making it a practical choice for cost estimation.

By comparing these diverse models, we aim to identify the most suitable approach for cost estimation in logic synthesis and potentially combine their strengths through ensemble methods.

3.2 Feature Engineering

The original E-Syn implementation primarily relies on basic features extracted from the circuit representation, including AST size (ASTSize), AST depth (ASTDepth), operator counts (+, !, *, &), Liberty cost metrics (SUM_LIB, AVE_LIB), and total node count (SUM_NODE). While these features provide fundamental information about the circuit structure, they may not fully capture the complexity and characteristics that influence post-mapping area and delay.

To enhance the predictive power of our regression models, we extend the feature set by incorporating additional feature categories. In total, we add **27 new features** across four categories. The complete list of these 27 features and their descriptions are presented in Table 1.

To identify which features are most critical for accurate cost estimation, we employ permutation importance analysis. This technique measures the importance of each feature by randomly permuting its values and observing the resulting increase in prediction error. Features that cause a significant increase in error when permuted are considered more important for

Table 1: New Features Added to E-Syn Cost Estimation Models

Category	Feature Name	Description
Structural Features (5)	num_internal_nodes	Number of internal nodes (non-leaf nodes) in the circuit, representing the complexity of the logic structure
	avg_fanin	Average number of input connections per node, indicating the average input dependency
	max_fanin	Maximum number of input connections for any single node, identifying highly connected nodes
	avg_fanout	Average number of output connections per node, showing how signals are distributed
	max_fanout	Maximum number of output connections for any single node, identifying nodes that drive many other nodes
Syntactic Features (11)	count_and	Total count of AND operators (& and *) in the circuit
	count_or	Total count of OR operators (+) in the circuit
	count_xor	Total count of XOR operators (^) in the circuit
	count_not	Total count of NOT operators (!) in the circuit
	total_operators	Sum of all operator counts, representing the total number of logic operations
	ratio_and	Proportion of AND operators relative to total operators
	ratio_or	Proportion of OR operators relative to total operators
	ratio_xor	Proportion of XOR operators relative to total operators
	ratio_not	Proportion of NOT operators relative to total operators
	num_parentheses	Number of parentheses pairs in the expression, indicating structural nesting
	max_nesting_depth	Maximum depth of nested expressions, measuring the deepest level of logic hierarchy
Expression Complexity Features (7)	avg_expr_length	Average length (number of nodes) of sub-expressions, measuring typical expression size
	max_expr_length	Maximum length of any sub-expression, identifying the largest expression component
	min_expr_length	Minimum length of any sub-expression, identifying the smallest expression component
	std_expr_length	Standard deviation of expression lengths, measuring variability in expression sizes
	avg_nesting_depth	Average nesting depth across all nodes, indicating typical hierarchy level
	min_nesting_depth	Minimum nesting depth, showing the shallowest hierarchy level
	std_nesting_depth	Standard deviation of nesting depths, measuring variability in hierarchy structure
Graph Features (4)	graph_nodes	Total number of nodes in the circuit graph representation
	graph_edges	Total number of edges (connections) in the circuit graph
	graph_density	Ratio of actual edges to maximum possible edges, measuring graph connectivity density
	avg_degree	Average degree (number of connections) per node in the graph

the model’s predictions. Through this analysis, we can identify the most influential features and potentially remove redundant or less informative features, leading to more efficient and accurate models.

4.1 Overall Performance

4.2 Comparison with Baseline Methods

4.3 Ablation Studies

4.4 Case Studies

4.5 Visualization

4 Experimental Results

5 Discussion

This section presents the experimental results and analysis.

This section discusses the results, insights, limitations, and future work.

5.1 Result Analysis

5.2 Key Insights

5.3 Limitations

5.4 Future Work

6 Conclusion

References

- [1] Chen Chen, Guangyu Hu, Dongsheng Zuo, Cunxi Yu, Yuzhe Ma, and Hongce Zhang, “E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis,” 2024.
- [2] Guangyu Hu, “E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis,” GitHub repository, 2024. [Online]. Available: <https://github.com/Gy-Hu/E-Syn>
- [3] Robert Brayton and Alan Mishchenko, “ABC: An Academic Industrial-Strength Verification Tool,” *Computer Aided Verification*, Springer, 2010, pp. 24–40.